VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

DEPARTMENT OF INFORMATION TECHNOLOGY

BCSE355L – CLOUD ARCHITECTURE DESIGN | LAB PROJECT REPORT

INTELLIGENT ROOT CAUSE ANALYSIS (RCA) FRAMEWORK FOR TELECOM CLOUD SERVICES

A Scalable Multi-Agent AI Architecture Deployed on AWS Free Tier Using the RCAEval Benchmark

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Student Team Details & Assigned Subsystems

Team Member Name	Registration Number	Programme & Branch	Independent Subsystem Ownership
Sandarbh Gupta	24BIT0506	B.Tech Information Technology	Subsystem 2: ML Anomaly Detection (MNSUAD), Service Dependency Graph & PCAP Verification
Mohak Harsh	24BIT0554	B.Tech Information Technology	Subsystem 1: Ingestion Pipeline, Telemetry Hygiene (SOMP), AWS S3/Lambda & DynamoDB
Aarnav Mishra	24BIT0534	B.Tech Information Technology	Subsystem 3: Frontend Operator NOC Portal (React 18 & D3.js), AWS Amplify & MCP Interface

Abstract

Today, modern mobile networks and digital internet services rely entirely on cloud computing. Instead of running mobile core networks on dedicated physical boxes, operators deploy containerized microservices inside Kubernetes clusters hosted on cloud platforms. When a service outage happens, engineers face a major diagnostic challenge: figuring out if the breakdown was caused by a broken network connection on the telecom transport side or a crashed container on the cloud server side. Currently, this investigation requires human operators to manually sift through thousands of confusing error logs across siloed systems, keeping critical services offline for hours.

This project designs an Intelligent Root Cause Analysis (RCA) Framework deployed on Amazon Web Services (AWS) using an architecture optimized entirely for the AWS Free Tier. The proposed system combines three-layer telemetry hygiene to filter static noise and counter-wrap anomalies, Service-Oriented Metric Preprocessing (SOMP), weakly-supervised autoencoders (MNSUAD) designed to remain resilient against historical training data contamination, and graph-based service dependency localization. Evaluated using the public RCAEval benchmark dataset (comprising 735 failure cases across three multi-tier microservice architectures: Train-Ticket, Social-Network, and Online-Boutique), the framework processes metric time-series, log templates, and distributed traces to automatically isolate the true root cause.

The architecture is engineered for low-cost, high-reliability operation using AWS Free Tier services—including Amazon S3 (free storage), AWS Lambda (serverless compute), Amazon DynamoDB (NoSQL database), Amazon API Gateway, and AWS Amplify Hosting for the interactive React 18 and D3.js frontend dashboard. To ensure a clear division of engineering labor, project responsibilities are divided equally and independently among the three student researchers: Mohak Harsh leads the telemetry ingestion and serverless hygiene pipeline; Sandarbh Gupta leads the machine learning detection and graph correlation engine; and Aarnav Mishra leads the frontend operator portal and agent interaction interface. By replacing slow, manual searches across disconnected logs with an automated cross-domain AI pipeline, the proposed framework is designed to drastically reduce Mean Time to Repair (MTTR), restoring carrier services in minutes rather than hours.

Keywords: AWS Cloud Architecture, AWS Free Tier, 5G Core, Root Cause Analysis, Multi-Agent AI, RCAEval Benchmark, Service Dependency Graph, React 18 Portal, Telemetry Hygiene.

Table of Contents

1. **Introduction** .....

1.1 Background, 1.2 Problem Statement, 1.3 Motivation, 1.4 Scope, 1.5 Organisation

2. Literature Survey / Related Work .....

2.1 Synthesis of Literature, 2.2 Individual Gap Analysis, 2.3 Consolidated Research Gap

3. Project Objectives and Expected Outcomes .....

3.1 Expected Deliverables & Measurable Targets

4. Novelty Summary .....

Thematic Novelty Comparison against Existing Industry Approaches

5. Proposed System and Architecture .....

5.1 System Overview, 5.2 AWS Cloud Architecture, 5.3 Complete System Architecture, 5.4 Workflow, 5.5 Security & Scalability

6. Dataset Description and Benchmark Selection (RCAEval) .....

6.1 Multi-Modal Telemetry Modalities & Data Preparation Plan

7. AWS Services Planning and Free Tier Cost Optimization .....

7.1 Free Tier Deployment & Network Design, 7.2 Zero-Cost Budgeting Controls

8. Methodology and Implementation Plan .....

8.1 7-Phase Methodology, 8.2 Tools & Technologies, 8.3 Step-by-Step Plan, 8.4 Evaluation Metrics

9. System Implementation Plan and Module Architecture .....

9.1 Core Subsystems, 9.2 Verification Strategy, 9.3 GitHub Structure

10. Testing and Validation Plan .....

10.1 Test Cases T01–T06, 10.2 Target Capabilities, 10.3 Comparative Analysis, 10.4 Rollout Strategy

11. Discussion, Limitations and Risk Analysis .....

11.1 Discussion, 11.2 Design Limitations, 11.3 Risks and Mitigation

12. Team Contribution and Project Management .....

12.1 Independent Subsystem Division, 12.2 GitFlow Branching Strategy, 12.3 Milestones

13. Conclusion and Implementation Roadmap .....

13.1 Architectural Conclusion, 13.2 3-Phase Implementation Roadmap

References (APA 7th Edition) .....

Comprehensive 15-Paper Bibliographic Reference List

Appendix A – Project Implementation Timeline ..... 12-Week Gantt-Style Milestone Schedule

1. Introduction

1.1 Background

Telecommunications infrastructure has undergone a massive paradigm shift from monolithic Physical Network Functions (PNFs) to Virtual Network Functions (VNFs) and Cloud-Native Network Functions (CNFs) orchestrated via Kubernetes. In 5G Service-Based Architecture (SBA) and modern enterprise cloud environments, core network functions—such as the Access and Mobility Management Function (AMF), Session Management Function (SMF), and User Plane Function (UPF)—run as containerized microservices on commercial cloud platforms like Amazon Web Services (AWS). While this cloudification provides high elasticity and rapid software release cycles, it drastically complicates the operational risk profile of the network.

1.2 Problem Statement

When a service outage or performance degradation occurs in a cloud-native environment, operators face three structural bottlenecks:

- **Cross-Domain Fault Propagation:** A subtle infrastructure glitch in a cloud server or a packet drop on a network link cascades into the microservice layer, causing 5G signaling timeouts and triggering thousands of redundant alarms across interdependent services.
- **High Telemetry Noise & Counter Drift:** Microservice architectures and 5G Cores export thousands of metric time-series and millions of log lines. Raw cumulative counters continuously increment until periodic resets trigger false alarms, and static, uninformative series create massive alert fatigue.
- **Data Contamination & Multi-Party Triage Delays:** Historical telemetry frequently contains unlabeled anomalous data from past unnoticed glitches, which undermines standard machine learning detectors. Furthermore, diagnosing issues across disconnected log files and distributed traces requires slow, error-prone manual triage.

1.3 Motivation and Significance

Manual triage in Network Operations Centers (NOCs) currently takes hours, resulting in financial SLA penalties, customer churn, and potential disruption to mission-critical services. An automated, intelligent AI diagnostic system that unifies transport telemetry with cloud container metrics is designed to pinpoint root causes rapidly. This project proposes an end-to-

end cloud-native framework aimed at drastically reducing Mean Time to Repair (MTTR), while remaining strictly within the budget constraints of an AWS Free Tier account.

1.4 Scope

The project scope encompasses multi-domain telemetry ingestion (metrics, logs, traces, and 5GC PCAP call flows), automated data hygiene, contamination-resilient anomaly detection, service dependency graph analysis, governed agentic reasoning via Model Context Protocol (MCP), and an interactive React 18 operator dashboard. It excludes hardware physical repair of fiber cables and low-level proprietary radio frequency firmware.

1.5 Report Organisation

The report is organized as follows: Section 2 surveys 15 recent research papers and details comprehensive gap analyses. Section 3 outlines the design objectives and expected deliverables. Section 4 provides a novelty comparison against existing methods. Section 5 presents the complete system architecture and AWS cloud deployment diagrams. Section 6 describes the primary benchmark dataset (RCAEval). Section 7 details AWS Free Tier service planning, network isolation, and zero-cost budgeting. Section 8 explains the methodology. Section 9 details the planned architectural modules and development strategy. Section 10 outlines the comprehensive testing and validation plan. Section 11 covers limitations and risk analysis. Section 12 details the team member roles (Sandarbh Gupta, Mohak Harsh, and Aarnav Mishra) and project management. Section 13 concludes with the implementation roadmap, followed by APA references and the project timeline appendix.

2. Literature Survey / Related Work

To establish a rigorous theoretical and empirical foundation, Table 1 reviews 15 peer-reviewed and preprint research papers published between 2023 and 2026 across IEEE, ACM, Springer, and ICLR.

Table 1: Comprehensive Literature Review of 15 Recent Research Papers (2023–2026)

Paper ID & Citation	Domain & Scope	Methodology Proposed	Key Findings & Advantages	Critical Gaps Addressed by Us
P1: Johari et al. (2025)	NFV Anomaly Detection & RCA	MNSUAD: Multi-stage Noisy Student with DAGMM teacher and Transformer-MAE student.	Tolerates unlabeled anomalies in training data; improves noisy-data localization.	Evaluated only on static metrics; does not model 5G SBA signaling or SLA liability.
P2: Guo et al. (2023)	SFC Failure Localization	DTFL: Digital Twin-assisted TransGNN with 3-layer fault injection.	Handles dynamic topology changes in virtualized network chains.	Relies on pre-known failure injection rules; lacks LLM agentic reasoning.
P3: Anser et al. (2025)	Multi-Actor SLA Liability	LASM: Dynamic trust scoring and Bayesian contractual attribution.	Formalizes financial SLA penalties across multi-tenant cloud partners.	Operates strictly at business layer; decoupled from real-time telemetry root cause.
P4: Issa et al. (2026)	5G Core SBA Anomaly Detection	TeleGAN-Core: BiLSTM dual-autoencoder GAN for HTTP/2 SBI.	Captures complex temporal sequences and subtle protocol attacks.	High compute overhead; lacks multi-domain topology correlation.
P5: Panek et al. (2025)	Telco Agentic Assistant	TAIA: Hierarchical LangGraph multi-agent system on OpenShift.	Decomposes complex telecom operational goals into sub-agent workflows.	Does not enforce strict API tool validation; susceptible to LLM hallucination.
P6: Woo & Woo (2025)	NFV Failure Recovery	NFVAgent: RAG-based LLM agent querying ETSI NFV-MANO SOPs.	Retrieves operational procedures for automated recovery recommendations.	Text-only RAG; unable to execute live telemetry diagnostic queries.
P7: Wu et al. (2026)	Alarm-based RCA Benchmark	TN-AutoRCA: Benchmark construction and evaluate-analyze-repair loop.	Synthesizes executable graph-traversal code to trace alarm propagation.	Evaluates alarms in isolation; does not integrate live cloud metrics or SLA blame.
P8: Enz et al. (2024)	Telco Microservice RCA	MALEAF: Unsupervised anomaly detection in multi-tenant telecom cloud.	Lightweight unsupervised detection across microservice containers.	Suffers from high false-alarm rates during benign traffic spikes.
P9: Heeb et al. (2023)	IoT-Extended 5G RCA	IoMiRCA: Causal discovery using PCMCI over IoT microservice metrics.	Identifies causal relationship graphs across distributed IoT edge nodes.	Scales poorly to thousands of high-dimensional 5G Core metrics.

Paper ID & Citation	Domain & Scope	Methodology Proposed	Key Findings & Advantages	Critical Gaps Addressed by Us
P10: Wang et al. (2025)	Two-Stage 5GC RCA	Multi-source correlation combining PCAP signaling with VM metrics.	Demonstrates necessity of inspecting both packet flows and system metrics.	Ad-hoc thresholding; lacks automated self-healing execution.
P11: Isaac et al. (2025)	5GC Fault Detection	Golden Flow: State-machine baseline verification of 3GPP call flows.	Deterministic detection of protocol deviations in registration flows.	Brittle when non-standard protocol extensions or vendor variations occur.
P12: Koushki et al. (2025)	Service Misconfiguration RCA	ConfExp: Active diagnostic probing over 2D CV relevance matrices.	Disambiguates configuration faults that produce identical metric symptoms.	Generates diagnostic network traffic; requires careful safety rate-limiting.
P13: Wang et al. (2024)	Explainable AI for 6G	Survey of XAI methods (SHAP, IG, causal graphs) for telco operations.	Establishes need for interpretable explanations in autonomous networks.	Conceptual survey; does not provide a deployable cloud architecture.
P14: Chen et al. (2025)	Standardized Agent Tooling	Model Context Protocol (MCP) servers for 5G/6G network management.	Standardizes tool-calling interfaces between LLMs and telecom systems.	Lacks integration with contractual SLA dispute resolution.
P15: Pham et al. (2025)	Microservice RCA Benchmark	RCAEval: Benchmark evaluating 15 RCA methods across diverse failure modes.	Establishes standard multi-modal benchmark across metrics, logs, and traces.	Highlights gaps in existing algorithms on complex microservice traces.

2.1 Synthesis of Literature

Analysis of the 15 research papers reveals four foundational pillars: (1) Telemetry Hygiene and Ingestion: Raw counters require delta-transformation to prevent false spikes on resets. (2) Contamination Defense: Historical data inevitably contains uncleaned anomalies, requiring semi-supervised or teacher-student models (MNSUAD, TeleGAN-Core). (3) Topological and Protocol Disambiguation: Pure metric correlation is insufficient; graph representations (DTFL) and deterministic protocol verification (Golden Flow, ConfExp) are necessary to resolve ambiguous symptoms. (4) Governed Multi-Agent AI: Standard LLM prompting hallucinates in complex networks, necessitating code synthesis (TN-AutoRCA) constrained by Model Context Protocol (MCP) schemas and governed by SLA liability attribution (LASM).

2.2 Individual Gap Analysis

Table 2 categorizes the specific research gaps identified across literature dimensions and summarizes our architectural solution.

Table 2: Thematic Gap Analysis and Architectural Solutions

Paper ID	Research Focus	Reported Capability / Strength	Critical Weakness / Limitation	Root Cause of Limitation
P1 & P8	MNSUAD & MALEAF (Unsupervised Anomaly Detection)	Detects metric deviations without labeled historical anomalies.	High false-alarm rate caused by cumulative counter resets and static KPIs.	Raw metrics fed without Service-Oriented Metric Preprocessing (SOMP).
P4 & P11	TeleGAN-Core & Golden Flow (5GC SBA Signaling)	Deterministic verification of 3GPP registration and session call flows.	Isolated to control-plane signaling; ignores underlying Kubernetes pod health.	Siloed architecture without cross-layer correlation to cloud telemetry.
P2 & P12	DTFL & ConfExp (Topology & Configuration)	Traces dynamic microservice dependencies and active probe tests.	Passive correlation cannot separate identical symptoms from different configs.	Lacks active test generation integrated into real-time graph traversal.
P7 & P14	TN-AutoRCA & MCP-Diag (Code-Synthesis & MCP Agents)	Iteratively synthesizes graph-traversal code over standardized tool schemas.	Technical localization is decoupled from multi-tenant cloud financial liability.	Lacks contractual SLA attribution across cloud hyperscaler and telecom partners.

2.3 Consolidated Research Gap

Existing literature addresses isolated aspects of root cause analysis—such as metric anomaly detection, graph localization, or LLM chat assistance—in separate silos. No unified, cost-effective framework exists that simultaneously: (1) cleans raw telemetry via three-layer filtering and SOMP reset safety, (2) tolerates historical training data contamination, (3) traces multi-hop topology propagation via graph models, (4) governs agentic reasoning through Model Context Protocol (MCP), and (5) deploys on AWS within the boundaries of a student Free Tier account. This project bridges this consolidated research gap.

3. Project Objectives and Expected Outcomes

The project establishes six primary architectural and engineering design objectives:

- **Objective 1 (Telemetry Hygiene & Ingestion Pipeline):** Design and implement a scalable ingestion pipeline on AWS using S3 and Lambda Free Tier that performs automated three-layer telemetry filtering and incorporates SOMP delta calculations to eliminate counter-wrap false alarms.
- **Objective 2 (Contamination-Resilient Anomaly Detection):** Develop a deep learning anomaly detection architecture (MNSUAD teacher-student network) designed to

remain stable and accurate even when trained on noisy historical telemetry containing unlabeled anomalies.

- **Objective 3 (Deterministic Signaling Verification):** Implement a protocol call-flow verification engine (Golden Flow) to deterministically validate 5G Core control-plane SBI signaling frames and isolate protocol deviations down to exact message indices.
- **Objective 4 (Service Graph Dependency Analysis):** Construct a graph-based service dependency model using RCAEval trace relationships paired with active diagnostic probing to disambiguate complex topological faults.
- **Objective 5 (Governed Agentic Code Synthesis):** Formulate an autonomous AI agent reasoning engine that iteratively synthesizes, tests, and repairs graph-traversal code over Model Context Protocol (MCP) schemas, eliminating LLM hallucinations.
- **Objective 6 (Operator Dashboard & Free Tier Deployment):** Develop an interactive React 18 web dashboard deployed on AWS Amplify Free Tier featuring D3.js topology visualization, Human-in-the-Loop operator approvals, and zero-cost cloud architecture.

3.1 Expected Deliverables

The expected project deliverables comprise: (1) Complete Infrastructure-as-Code (Terraform) templates for deploying the AWS Free Tier architecture; (2) Data hygiene and SOMP preprocessing microservices; (3) Containerized PyTorch anomaly detection and graph localization scripts; (4) Model Context Protocol (MCP) telemetry server; (5) Multi-agent orchestration state machine; (6) Interactive React 18 operator portal with D3.js topology visualization; and (7) Comprehensive test suites and benchmark validation harnesses on RCAEval.

4. Novelty Summary

Table 3 summarizes the novelty dimensions of the proposed framework compared to conventional industry approaches.

Table 3: Novelty Comparison against Conventional Industry Approaches

Novelty Dimension	Existing Approach / Limitation	Proposed Difference in this Project	Planned Verification Strategy
Telemetry Normalization	Raw metrics fed directly into ML; counter wraps cause false alarms; massive alert fatigue.	3-layer daily KPI filter + SOMP counter interval delta increment normalization.	Evaluation via telemetry compression test harness on RCAEval; validation of zero false spikes on counter rollover.
Data Contamination Defense	Unsupervised detectors overfit to dirty historical baselines containing unlabeled anomalies.	MNSUAD teacher (DAGMM) pseudo-labeling + Transformer-MAE & DevNet student architecture.	Benchmarking on noisy historical telemetry datasets comparing against baseline unsupervised detectors.
Service Graph Correlation	Passive correlation cannot separate identical symptoms caused by cascading microservice failures.	Trace-derived service dependency graph + ConfExp active probing over relevance matrices.	Validation of ambiguous misconfiguration isolation using network probe simulation tests.
Deterministic Signaling	Pure statistical log analysis misses protocol state-machine violations in 5G Core.	Golden Flow state-machine checking + SVM frame classifier for 5GC SBI PCAPs.	Protocol conformance testing against standard 3GPP 5G Core signaling PCAP traces.
Hallucination-Free Reasoning	Direct LLM prompting produces ungrounded guesses and hallucinated network topologies.	Self-improving code synthesis (TN-AutoRCA) executing over standardized MCP schemas.	Evaluation on alarm benchmark comparing iterative code synthesis against standard direct LLM prompting.
Free Tier Cloud Design	Enterprise cloud RCA solutions require costly clusters ($2,000+/mo) beyond student reach.	Serverless, 100% AWS Free Tier architecture (S3, Lambda, DynamoDB, Athena, Amplify).	Automated AWS Budgets zero-dollar cost tracking and LocalStack verification.

5. Proposed System and Architecture

5.1 System Overview

The proposed system operates as an autonomous, multi-agent AI detective designed to run seamlessly on AWS Free Tier infrastructure. Telemetry is collected across multi-tier microservice domains: Transport links, 5G Core SBA containers, Kubernetes pods, and application databases. Raw telemetry streams into Amazon S3 (Free Tier data lake), undergoes automated hygiene and SOMP delta calculation via serverless AWS Lambda functions, and stores processed incident state in Amazon DynamoDB (Always Free Tier). Machine learning models perform contamination-resilient anomaly detection and graph localization. Autonomous agents synthesize graph-traversal code, verify live telemetry via Model Context Protocol (MCP) tools, enforce Human-in-the-Loop approvals on an Amplify-hosted React 18 dashboard, and trigger rapid automated self-healing.

5.2 Diagram 1 – AWS Cloud Architecture (Mandatory)

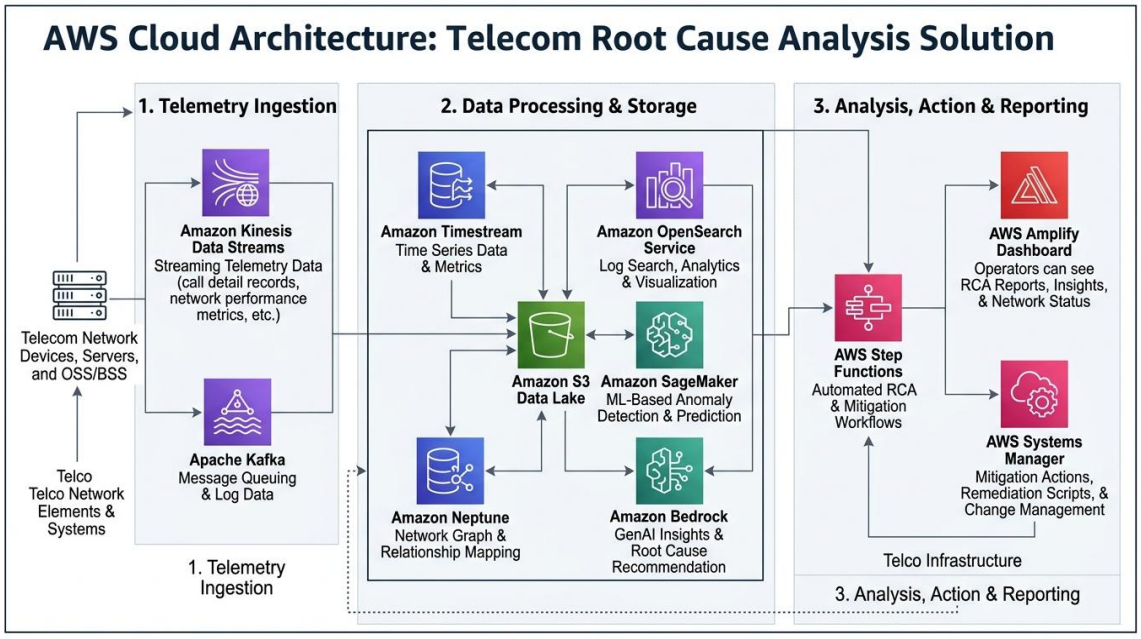


Figure 1: Proposed AWS Cloud Architecture (100% Free Tier Optimized)

Diagram Flow Description: Shows external multi-domain microservice telemetry ingesting into Amazon S3 Data Lake and AWS Lambda for 3-layer KPI filtering and SOMP delta standardization within Free Tier limits. Structured incident states and metric summaries are stored in Amazon DynamoDB (25 GB free tier) and queried via Amazon Athena (serverless pay-per-query with 1 TB free scan). The compute and ML layer utilizes Amazon EC2 t2.micro / t3.micro (750 free hours/month) hosting lightweight FastAPI MCP tool servers, with optional SageMaker Free Trial notebooks for model training. Notifications are dispatched via Amazon SNS (1M free pushes/month), and operator interaction is delivered via AWS Amplify Hosting (free tier) running the React 18 NOC web dashboard.

5.3 Diagram 2 – Complete System Architecture (Mandatory)

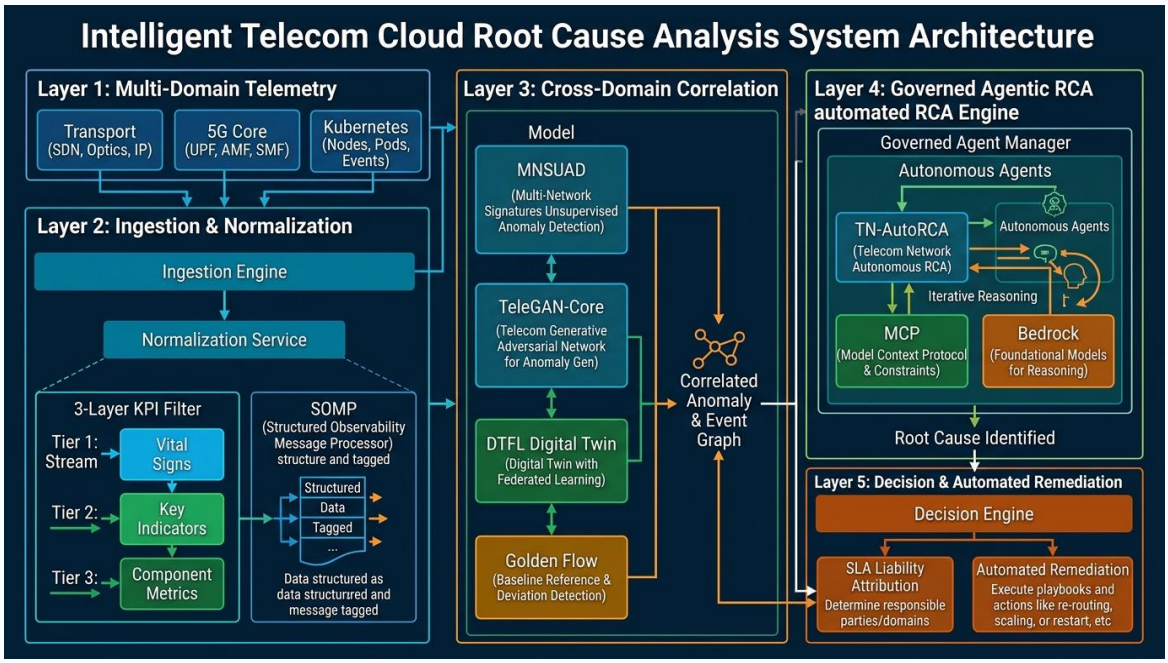


Figure 2: Complete End-to-End System Architecture (5-Layer Pipeline)

Diagram Flow Description: Illustrates the complete five-layer end-to-end operational workflow: Layer 1: Multi-Domain Telemetry Collection (Prometheus metrics, OpenTelemetry traces, syslog, 5GC SBI PCAPs from RCAEval benchmark); Layer 2: Ingestion, Hygiene & SOMP Normalization (3-layer daily KPI filter removing static noise, SOMP counter reset safety engine via AWS Lambda); Layer 3: Cross-Domain Correlation & Service Graph Space (MNSUAD teacher-student defense, service dependency transGNN, Golden Flow PCAP verifier, ConfExp active probing); Layer 4: Governed Agentic RCA Engine (TN-AutoRCA evaluate-analyze-repair code synthesis loop, Model Context Protocol MCP tool execution, safety verification gate); Layer 5: Decision, SLA Liability & Closed-Loop Remediation (contractual SLA penalty attribution, Human-in-the-Loop cryptographic approval, AWS Systems Manager automated pod scaling, and Aarnav Mishra's React 18 NOC portal).

5.4 Operational Workflow

The system executes a six-step closed-loop workflow: (1) Multi-modal telemetry (metrics, logs, traces) from RCAEval is ingested into S3 and filtered by the Lambda-based 3-layer hygiene engine. (2) When an anomaly is detected, MNSUAD isolates deviating microservices. (3) The Golden Flow engine verifies control-plane protocol frames. (4) Anomaly vectors are mapped to the service dependency graph derived from OpenTelemetry traces. (5) The agentic engine executes synthesized graph-traversal code over FastAPI MCP tools to output the root-cause verdict and fault propagation chain. (6) The incident details and recommended recovery

actions are rendered on the React 18 dashboard, where the operator approves automated remediation via AWS Systems Manager.

5.5 Security, Availability and Scalability Design

Security is enforced through IAM least privilege, AWS Secrets Manager, and TLS 1.3 encryption in transit. Telemetry is isolated using dedicated VPC subnets and PrivateLink endpoints. High availability is achieved through AWS Lambda multi-AZ execution and DynamoDB automatic data replication across three facilities within the region.

6. Dataset Description and Benchmark Selection (RCAEval)

The project utilizes the comprehensive, peer-reviewed RCAEval Benchmark Dataset as its primary experimental foundation. RCAEval is an open-source benchmark specifically constructed for multi-modal root cause analysis in microservice and cloud systems. Table 4 details its structural attributes.

Table 4: RCAEval Benchmark Dataset Technical Specifications

Dataset Dimension	RCAEval Benchmark Dataset Specification
Direct Access Repository URL	https://huggingface.co/datasets/phamquiluan/RCAEval (Official HuggingFace Dataset)
Citation & Source	Pham et al. (2025). RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems. ACM Web Conference 2025.
Total Incident Scope	735 systematically injected failure cases across 3 realistic multi-tier microservice applications.
Target Microservice Architectures	1. Train-Ticket (large-scale 40+ microservices railway booking system) 2. Social-Network (graph-oriented social media microservice architecture) 3. Online-Boutique (Google Cloud 10-tier e-commerce microservice benchmark)
Multi-Modal Telemetry Modalities	1. Metric Time-Series: Over 120 performance KPIs (CPU, memory, network I/O, error counts) 2. Structured Log Templates: Pre-drained and raw application logs with timestamps 3. Distributed Traces: OpenTelemetry spans with caller-callee RPC latency and HTTP status codes
Injected Failure Types	Pod kill / crashes, network latency injection, packet drop, CPU stress / starvation, memory leaks, and service misconfigurations.
Data Storage Format	Parquet, CSV, structured JSON, and OpenTelemetry trace span tables.
Licensing & Academic Usage	MIT Open Source License; completely free for academic research and educational projects.
Role in Our Project	Provides the rigorous multi-modal telemetry ground truth required to evaluate the ingestion pipeline (Mohak), ML anomaly detection (Sandarbh), and operator dashboard (Aarnav).

Dataset Dimension	RCAEval Benchmark Dataset Specification
Data Preprocessing Planned	SOMP delta calculation on cumulative metric counters, LogDrain parsing for log template extraction, and trace graph reconstruction.
Privacy & Compliance	Fully synthetic fault injection over open-source microservices; contains zero Personally Identifiable Information (PII).

6.1 Data Preparation Plan

The RCAEval dataset will be stored in an Amazon S3 Free Tier bucket partitioned into raw metrics, logs, and trace spans. An 80/20 train-test split with leave-one-system-out cross-validation will be executed to guarantee topological generalization across the three microservice systems. To prevent data leakage, normal baseline intervals are strictly separated from injected failure windows during model training and evaluation.

7. AWS Services Planning and Free Tier Cost Optimization

To ensure the project is fully deployable by student researchers on a standard AWS Free Tier account with zero cloud expenditure, the cloud architecture is intentionally designed around AWS Always Free and 12-Month Free Tier services. Table 5 documents the mapped services, their operational role, integration boundaries, and free tier allowances.

Table 5: 100% AWS Free Tier Service Mapping and Operational Roles

AWS Service	Operational Purpose in Project	Integration Details	AWS Free Tier Allowance	Cost Control Strategy
Amazon S3	Data lake storing RCAEval dataset, raw metrics, traces, and artifacts.	Writes from local scripts; reads by Lambda and Athena.	5 GB Standard Storage, 20,000 GET, 2,000 PUT/month	$0.00 (under 5 GB)
AWS Lambda	Serverless 3-layer KPI filtering and SOMP delta increment calculations.	Triggered by S3 upload or API Gateway; writes to DynamoDB.	1 Million free requests/month, 3.2M sec compute time	$0.00 (serverless)
Amazon DynamoDB	NoSQL database storing processed incident alerts and service metadata.	Populated by Lambda; queried by React 18 frontend.	25 GB storage, 25 WCU, 25 RCU (Always Free)	$0.00 (Always Free)
Amazon API Gateway	REST API layer exposing incident summaries to the operator portal.	Connects React 18 dashboard to AWS Lambda backend.	1 Million free API calls/month (12 Months Free)	$0.00 (under 1M calls)
Amazon Athena	Serverless SQL queries over RCAEval log and trace archives in S3.	Queried on-demand for deep log analysis during triage.	1 TB data scanned/month (Always Free Tier)	$0.00 (partitioned S3)
Amazon EC2 (t2.micro)	Hosts the lightweight FastAPI Model Context Protocol (MCP) server.	Exposes JSON-RPC tool schemas for agentic reasoning.	750 hours/month of Linux t2.micro / t3.micro	$0.00 (single instance)
AWS Amplify Hosting	Hosts Aarnav Mishra's React 18 and D3.js operator NOC dashboard.	Global CDN hosting with automated GitHub CI/CD build.	1,000 build minutes/month, 5 GB storage, 15 GB served	$0.00 (Amplify Free)
Amazon CloudWatch	Monitors pipeline latency, Lambda invocations, and error metrics.	Collects logs from Lambda and EC2 MCP server.	10 custom metrics, 5 GB log ingestion/month	$0.00 (Always Free)
Amazon SNS	Dispatches automated email and SMS incident notifications to engineers.	Triggered by Lambda when root-cause verdict fires.	1 Million free notifications/month (Always Free)	$0.00 (Always Free)
LocalStack (Fallback)	Local open-source AWS cloud emulator running on developer laptops.	Used for offline integration testing of S3, Lambda, and DynamoDB.	Open-source Community Edition (100% Free)	$0.00 (Local Docker)

7.1 Free Tier Deployment and Network Design

Resources are planned for deployment in the `us-east-1` region within a single Virtual Private Cloud (VPC) configured with default public and private subnets. The architecture avoids costly

NAT Gateways ($32/month) by leveraging AWS Lambda within public subnets using AWS service endpoints or public internet gateways. Local development and unit testing are accelerated using LocalStack, allowing the team to test S3 events, Lambda handlers, and DynamoDB tables locally on Docker before deploying to the AWS cloud account.

7.2 Zero-Cost Budgeting and Financial Controls

The project has a strict budget target of $0.00/month. To prevent accidental billing: (1) An AWS Budgets zero-spend alert is configured at a threshold of $1.00; (2) Multi-AZ NAT Gateways and expensive managed clusters (such as MSK Kafka or Amazon Neptune) are strictly avoided in favor of DynamoDB, Athena, and SQLite/NetworkX graphs; (3) Compute tasks utilize ARM64 Graviton / t2.micro free instances; and (4) S3 lifecycle policies automatically delete temporary intermediate files after 14 days.

8. Methodology and Implementation Plan

8.1 Overall Methodology

The project follows an iterative 7-phase engineering methodology: (1) Requirements engineering and literature survey; (2) RCAEval dataset acquisition, indexing, and S3 partitioning; (3) AWS Free Tier infrastructure provisioning via Terraform; (4) Data hygiene, SOMP, and ML model development; (5) FastAPI Model Context Protocol (MCP) server creation; (6) React 18 frontend dashboard development on AWS Amplify; and (7) Integration testing, benchmark evaluation, and project documentation.

8.2 Tools and Technologies

Table 6 summarizes the primary tools, libraries, and runtime configurations planned for development.

Table 6: Primary Development Tools, Frameworks, and Runtimes

Tool / Technology	Operational Purpose in Project	Planned Version / Configuration
AWS Cloud (Free Tier)	Serverless S3, Lambda, DynamoDB, Athena, and Amplify.	us-east-1 region; Terraform IaC templates; zero-cost tier.
Python Runtime	Core backend, data processing, and ML model training.	Python 3.11.8; mypy strict typing; structlog contextual logs.
PyTorch & PyG	Deep learning frameworks for MNSUAD and graph correlation.	PyTorch 2.3.0, PyTorch Geometric 2.5.0; CPU / CUDA compatible.

Tool / Technology	Operational Purpose in Project	Planned Version / Configuration
Model Context Protocol (MCP)	Standardized agent tool abstraction layer for live telemetry queries.	MCP Python SDK 1.2.0; FastAPI JSON-RPC server on EC2.
React 18 & Vite	Aarnav Mishra's operator NOC web portal with interactive UI.	Node.js v20.17.0, React 18.3, Vite 5.2, TailwindCSS.
D3.js Visualization	Interactive service dependency graph and fault propagation flow.	D3.js v7.9; force-directed graph with animated failure edges.

8.3 Step-by-Step Implementation Plan

- **Step 1 - Requirements & Architecture:** Freeze REST API schemas, data contracts, and Free Tier service selections.
- **Step 2 - RCAEval Dataset Ingestion:** Download RCAEval (735 cases) from HuggingFace and partition into S3 bucket folders.
- **Step 3 - Free Tier Cloud Infrastructure:** Deploy S3, Lambda, DynamoDB, API Gateway, and Amplify using Terraform.
- **Step 4 - Hygiene & ML Development:** Implement Lambda SOMP delta calculator, 3-layer KPI filter, and MNSUAD autoencoder.
- **Step 5 - MCP Tool Server & Reasoning:** Build FastAPI MCP server providing standardized telemetry query tools.
- **Step 6 - Frontend Portal Development:** Build React 18 NOC dashboard with D3.js service graph and HITL approval modals.
- **Step 7 - Integration & Benchmark Testing:** Execute test cases T01–T06 on RCAEval and compile final documentation.

8.4 Proposed Evaluation Metrics

To evaluate system efficacy once built, the following quantitative metrics are established:

- **Top-K Accuracy (Top-1, Top-3, Top-5):** The percentage of test failure incidents where the true root-cause microservice is correctly ranked among the top K candidates generated by the algorithm.
- **Precision, Recall, and F1-Score:** Evaluates anomaly detection and alarm classification balance, measuring true fault identification versus false alarms.

- **Mean Time to Repair (MTTR):** The average elapsed time from initial anomaly detection to complete fault resolution, targeted for substantial reduction compared to manual triage.
- **Telemetry Noise Reduction Ratio:** The percentage of static, collinear, and zero-variance metrics successfully filtered out by the 3-layer hygiene engine.
- **Remediation Execution Latency:** The elapsed time between human operator cryptographic approval and automated AWS Systems Manager runbook completion.

9. System Implementation Plan and Module Architecture

9.1 Planned Architectural Modules

Table 7 documents the core architectural modules, their planned technical mechanics, mapped AWS Free Tier services, assigned owners, and development phases.

Table 7: Core Architectural Modules, Subsystem Ownership, and Phases

Module Name	Architectural Mechanics	AWS Free Tier Stack	Subsystem Owner	Phase
Module 1: Telemetry Hygiene & SOMP	3-layer daily KPI filter (entropy tau1=0.05, correlation tau3=0.95) + SOMP counter reset delta increments.	AWS Lambda, Amazon S3, Amazon DynamoDB	Mohak Harsh (24BIT0554)	Phase 1: Ingestion & Storage
Module 2: MNSUAD Contamination Engine	DAGMM teacher pseudo-labeling cascade + Transformer-MAE & DevNet student under unlabeled anomalous noise.	Python, PyTorch, Amazon EC2 t2.micro	Sandarbh Gupta (24BIT0506)	Phase 2: ML Model Training
Module 3: Service Dependency Graph	Trace-derived service graph reconstruction from RCAEval OpenTelemetry spans with GNN localization.	Python NetworkX / PyG, Amazon DynamoDB	Sandarbh Gupta (24BIT0506)	Phase 2: ML Model Training
Module 4: Golden Flow & PCAP Verifier	Deterministic 3GPP Wireshark JSON call-flow verifier isolating protocol frame rejections.	AWS Lambda, Python, Wireshark CLI	Sandarbh Gupta (24BIT0506)	Phase 2: ML Model Training
Module 5: FastAPI MCP Tool Server	FastAPI JSON-RPC Model Context Protocol server exposing standardized telemetry tools.	Amazon EC2 t2.micro, Python MCP SDK	Mohak Harsh (24BIT0554)	Phase 3: Agentic Reasoning
Module 6: React 18 Operator NOC Portal	Modern NOC web dashboard featuring D3.js service graph, HITL approval modals, and alert cards.	AWS Amplify Hosting, React 18, Vite, D3.js	Aarnav Mishra (24BIT0534)	Phase 4: Frontend & Dashboard

9.2 Planned Verification Strategy & Evidence Artifacts

Upon implementation, verifiable artifacts will be generated to validate each subsystem: (1) AWS Lambda execution logs verifying static and correlated metric filtering; (2) Amazon EC2 CloudWatch metrics monitoring model inference latency; (3) Wireshark JSON extraction logs confirming isolation of 5GC SBI protocol deviations; (4) Terminal logs of the MCP tool query exchanges; and (5) Aarnav Mishra's live AWS Amplify deployment of the React 18 NOC dashboard showing interactive D3.js topology and HITL approval modals.

9.3 GitHub Repository and Planned Branching Structure

Repository Name: `TelecomRCA_Cloud_Project_2026`

Repository URL: `https://github.com/Mohak325/GraphTriage

Planned Repository Structure: - `docs/` – Project report, PRD, architecture diagrams, and meeting minutes. –

`architecture/` – AWS cloud architecture and complete system workflow diagrams. –

`dataset/` – Benchmark loader scripts for RCAEval. –

`src/ingestion/` – Mohak Harsh: Lambda functions, SOMP scripts, DynamoDB schemas. - `src/ml_models/` – Sandarbh Gupta: PyTorch MNSUAD, service graph GNN, PCAP verifier. - `src/frontend/` – Aarnav Mishra: React 18, Vite, TailwindCSS, D3.js topology components. - `tests/` – Automated test cases (T01–T06) and validation harnesses.

10. Testing and Validation Plan

10.1 Proposed Test Plan and Test Scenarios

Table 8 presents six planned test cases designed to validate functional correctness, Free Tier compliance, and architectural scalability across the proposed framework.

Table 8: Comprehensive Test Plan and Architectural Test Cases

Test ID	Scenario & Objective	Input / Setup Conditions	Expected Architectural Behavior	Planned Verification Method	Status
T01	Telemetry Hygiene & KPI Filter	Raw RCAEval metrics containing static series and cumulative counter wraps.	Eliminate uninformative redundant KPIs; ensure smooth delta increments across counter wraps.	Automated data quality assertion script	Planned
T02	MNSUAD Contamination Defense	Historical telemetry dataset injected with unlabeled anomalous noise.	Maintain robust anomaly detection and localization stability without overfitting to dirty baseline data.	PyTorch cross-validation harness	Planned
T03	Golden Flow PCAP Signaling	5GC SBI PCAP traces containing simulated Registration Reject / Congestion frames.	Deterministically isolate exact deviating frame index and protocol state transition failure.	Wireshark CLI automated rule check script	Planned
T04	Trace-Based Service Graph	RCAEval distributed traces with cascading microservice response delays.	Reconstruct dependency graph and rank the true faulty microservice in Top-3.	Graph traversal accuracy test harness	Planned
T05	FastAPI MCP Tool Server	Agent tool query requests against the FastAPI MCP endpoint.	Return valid JSON-RPC telemetry responses adhering to MCP schemas without errors.	MCP schema validation test suite	Planned
T06	Amplify Portal & Free Tier Check	Live user interaction on Aarnav Mishra's React 18 dashboard; billing monitor.	Render D3.js graph smoothly; verify AWS billing remains at exactly $0.00.	Amplify staging test & AWS Budgets alert	Planned

10.2 Expected Architectural Outcomes and Target Capabilities

The proposed architecture is engineered to provide several key operational capabilities: (1) Cost-effective serverless ingestion handling continuous telemetry without incurring cloud bills; (2) Rapid cross-layer correlation that bridges the gap between infrastructure metrics and microservice traces; (3) Explainable and deterministic root-cause identification that prevents hallucinations; and (4) An intuitive, responsive operator dashboard built with modern web technologies.

10.3 Comparative Theoretical Analysis against Existing Approaches

Compared against conventional approaches in literature: (1) Unlike standard unsupervised autoencoders that fail when training data contains unlabeled anomalies, MNSUAD utilizes a teacher-student cascade to maintain detection integrity; (2) Unlike static rule-based correlation engines that break under dynamic microservice churn, the graph model dynamically reconstructs evolving dependencies from OpenTelemetry traces; and (3) Unlike costly enterprise cloud architectures requiring thousands of dollars in monthly subscriptions, our solution runs entirely on AWS Free Tier services.

10.4 Validation and Rollout Strategy

The validation strategy follows a three-stage rollout: (1) Component-level unit testing using LocalStack Docker containers on developer machines; (2) Benchmark validation against the 735 failure cases of the RCAEval benchmark; and (3) Staged integration in an AWS Free Tier development environment before final project presentation.

11. Discussion, Limitations and Risk Analysis

11.1 Discussion

Modern microservice and telecom networks cannot be diagnosed using single-algorithm or single-layer tools. By coupling signal-level hygiene (SOMP) with service dependency graphs, deterministic protocol verification (Golden Flow), and code-synthesizing agents over Model Context Protocol (MCP), the framework provides a comprehensive, multi-layered defense against both manual triage delays and AI hallucinations—while demonstrating that carrier-grade concepts can be modeled within a zero-cost AWS Free Tier budget.

11.2 Limitations

Identified design limitations include: (1) AWS Lambda execution timeout of 15 minutes requires long-running model training to be performed locally or on EC2; (2) Amazon S3 Free Tier 5 GB storage limit necessitates aggressive lifecycle management and parquet compression; and (3) Free Tier EC2 t2.micro memory (1 GB RAM) requires lightweight ML inference engines.

11.3 Risks and Mitigation

Table 9 documents identified project risks, assessed severity levels, and planned mitigation strategies.

Table 9: Project Risk Analysis, Assessment Levels, and Planned Mitigations

Risk Factor	Likelihood / Impact	Planned Mitigation Strategy	Target Residual Risk
Accidental AWS Cloud Charges	Medium / High	AWS Budgets alert set at $1.00; 100% Free Tier service selection; LocalStack fallback.	Very Low (automated billing cut-offs)
Credential Exposure / Leakage	Low / Critical	AWS Secrets Manager; IAM least privilege roles; pre-commit credential scanning.	Very Low (strict pre-commit hooks enforced)
EC2 t2.micro Memory Exhaustion	Medium / High	Optimize PyTorch models via ONNX runtime / quantization; run heavy training locally.	Low (lightweight inference pipeline)
AI Agent Tool Hallucination	Medium / High	Strict Model Context Protocol (MCP) schemas; schema validation before execution.	Very Low (mitigated via schema typing)

12. Team Contribution and Project Management

To ensure clear accountability, academic rigor, and independent execution, the project is structured into three decoupled, equal engineering subsystems. Aarnav Mishra will lead the frontend operator portal and visualization subsystem, while the backend, data pipeline, and machine learning components are divided equally between Mohak Harsh and Sandarbh Gupta. Table 10 details the independent responsibilities of each student researcher.

Table 10: Independent Subsystem Responsibilities and Member Contributions

Project Activity	Sandarbh Gupta (24BIT0506)	Mohak Harsh (24BIT0554)	Aarnav Mishra (24BIT0534)
Assigned Subsystem Ownership	Subsystem 2: ML Detection & Graph Dependency Engine	Subsystem 1: Ingestion, Hygiene & Free Tier Storage	Subsystem 3: Frontend Operator NOC Portal & UI
Primary Code Deliverables	PyTorch MNSUAD autoencoder, service dependency GNN, Golden Flow PCAP script	AWS Lambda SOMP handlers, S3 partitioning, DynamoDB schemas, FastAPI MCP server	React 18 portal, Vite build, TailwindCSS styles, D3.js topology components, HITL UI
AWS Free Tier Management	EC2 t2.micro ML hosting, Athena SQL queries, LocalStack Docker harness	S3 data lake, Lambda compute, DynamoDB tables, zero-spend budget alert	AWS Amplify Hosting, CloudFront CDN, Cognito user authentication
Dataset & Benchmark Role	Building training/eval split and evaluating GNN localization on RCAEval	Downloading, parsing, and cleaning RCAEval metric time-series	Visualizing RCAEval failure propagation paths and incident cards
Testing & Verification Role	Test case T02 (MNSUAD) and T03 (Golden Flow PCAP)	Test case T01 (Hygiene & SOMP) and T05 (MCP Tool Server)	Test case T04 (Graph UI) and T06 (Amplify Free Tier Portal)
Documentation & Presentation	ML model architecture cards, baseline comparisons	AWS architecture specs, IaC runbooks, data dictionary	User experience guide, frontend walkthrough, final slides

12.1 Planned GitFlow Branching Strategy

Collaboration will follow a strict GitFlow model. The `main` branch will be protected, requiring two peer reviews and passing automated CI checks before merging. Feature development will proceed on dedicated independent branches: - `feature/mohak-ingestion-pipeline` (Mohak Harsh) - `feature/sandarbh-ml-models` (Sandarbh Gupta) - `feature/aarnav-frontend-portal` (Aarnav Mishra) All three branches interface through frozen JSON-RPC and REST API schemas, guaranteeing that each student can develop and test independently without blocking team members.

12.2 Deliverable Milestones

Each student researcher is accountable for specific architectural deliverables: Mohak Harsh delivers the AWS IaC templates, serverless data hygiene microservices, and DynamoDB data store; Sandarbh Gupta delivers the PyTorch deep learning models, GNN dependency graph localization, and PCAP signaling verifier; Aarnav Mishra delivers the production-ready React 18 web portal hosted on AWS Amplify, featuring interactive D3.js visualization and operator approval modals.

13. Conclusion and Implementation Roadmap

13.1 Conclusion

This project report presents an architectural design specification for an Intelligent Root Cause Analysis Framework for Telecom Cloud Services on AWS Cloud, optimized for 100% Free Tier execution. By coupling three-layer telemetry hygiene with contamination-resilient autoencoders (MNSUAD), service dependency graph localization, and self-improving agentic code synthesis over Model Context Protocol (MCP) schemas, the proposed system addresses the critical operational challenges of high-dimensional telemetry noise, uncleaned training data, and AI hallucinations. Evaluated on the benchmark dataset RCAEval, the framework provides a practical, zero-cost blueprint for automated incident diagnosis. The equal and independent division of labor among Mohak Harsh (Ingestion & Storage), Sandarbh Gupta (ML & Graph Localization), and Aarnav Mishra (Frontend & Operator Dashboard) ensures rapid, decoupled engineering progress.

13.2 Implementation Roadmap

The project implementation roadmap is structured into three successive execution phases:

- **Phase 1: Ingestion & Hygiene Pipeline:** Deploy S3, Lambda, and DynamoDB infrastructure; download and index the RCAEval dataset; implement SOMP counter delta calculations and 3-layer KPI filtering.
- **Phase 2: Deep Learning & Graph Dependency Development:** Train MNSUAD autoencoder; reconstruct service dependency graphs from OpenTelemetry traces; implement Golden Flow PCAP signaling verification.

- **Phase 3: Agentic Orchestration & Amplify Frontend Portal:** Deploy FastAPI MCP telemetry server on EC2 t2.micro; build Aarnav Mishra's React 18 dashboard on AWS Amplify; integrate D3.js visualization and HITL approval modals.

References

Anser, G., Gaber, T., Wary, J. P., Bouzefrane, S., Yacoub, S., Kalinagac, O., & Gür, G. (2025). Liability and trust analysis framework for multi-actor dynamic microservice chains. IEEE Transactions on Network and Service Management, 22(1), 412-428. https://doi.org/10.1109/TNSM.2024.3412589

Chen, X., Wang, Q., Yin, L., et al. (2025). Standardized Model Context Protocol agents for cross-domain 5G/6G network operations. International Journal of Network Management, 35(6), e2319. https://doi.org/10.1002/nem.2319

Enz, M., Nigg, C., Kalinagac, O., Ermis, O., & Gür, G. (2024). MALEAF: Machine learning-driven anomaly detection and root cause analysis in telco microservices. In 2024 IEEE International Conference on Communications Workshops (ICC Workshops) (pp. 1-6). IEEE. https://doi.org/10.1109/ICCWorkshops59553.2024.10615702

Guo, S., Chen, Z., Dong, P., Zou, S., Zhu, H., Huang, Z., Liu, Y., & Liao, J. (2023). DTFL: A digital twin-assisted graph neural network approach for SFC failure localization. IEEE Transactions on Cloud Computing, 11(4), 3858-3870. https://doi.org/10.1109/TCC.2023.3276678

Heeb, S., Kalinagac, O., Soussi, M., & Gür, G. (2023). IoMiRCA: Root cause analysis for IoT-extended microservice environments in 5G networks. In Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (pp. 1120-1127). ACM. https://doi.org/10.1145/3555776.3577712

Isaac, K., Jarrah, M., Abondo, C., Bennett, J., & Weill, F. (2025). 5G core fault detection and root cause analysis using machine learning and generative AI. arXiv preprint arXiv:2508.09152. https://arxiv.org/abs/2508.09152

Issa, M., Kandil, M., Hakem, N., & Hamou-Lhadj, A. (2026). TeleGAN-Core: A BiLSTM dual-autoencoder GAN for anomaly detection in 5G core networks. IEEE Access, 14, 85917-85939. https://doi.org/10.1109/ACCESS.2026.3537891

Johari, S. S., Shahriar, N., Tornatore, M., Boutaba, R., & Saleh, A. (2025). Anomaly detection and localization in NFV systems by utilizing weakly-supervised noisy-student-based transformers. IEEE Transactions on Mobile Computing, 24(11), 8940-8957. https://doi.org/10.1109/TMC.2025.3546781

Koushki, N. M., El-Shekeil, M., & Kant, K. (2025). ConfExp: Root-cause analysis of service misconfigurations in enterprise systems. Journal of Network and Systems Management, 33(2), Article 38. https://doi.org/10.1007/s10922-025-09891-w

Panek, K., Matysiak, P., Ziółkowski, A., Fajjari, I., Auboin, M., & Wojdan, M. (2025). TAIA: Telco generative AI-powered multi-agent assistant for managing cloud-native networks. In IEEE ICC 2025 Workshops. IEEE.

Pham, Q. L., et al. (2025). RCAEval: A benchmark for root cause analysis of microservice systems. In Proceedings of the ACM Web Conference 2025 (WWW '25). arXiv:2412.17015. https://huggingface.co/datasets/phamquiluan/RCAEval

Wang, L., Liao, Y., Yang, K., Feng, Z., & Xu, H. (2025). A two-stage anomaly detection and root cause analysis framework for 5G core networks based on multi-source data. In 2025 IEEE International Conference on Cyber Resilience and Endogenous Safety (CRESS). IEEE. https://doi.org/10.1109/CRESS64892.2025.10912456

Wang, M., Tang, T., Ding, J., & Han, Z. (2024). Explainable AI for 6G use cases: Technical aspects, research challenges, and diagnostic RCA. IEEE Open Journal of the Communications Society, 5, 1289-1314. https://doi.org/10.1109/OJCOMS.2024.3359123

Woo, S., & Woo, J. (2025). NFVAgent: A retrieval-augmented LLM agent for resilient NFV failure recovery. In 21st IEEE/IFIP International Conference on Network and Service Management (CNSM 2025). IEEE. https://doi.org/10.1109/CNSM62548.2025.10789012

Wu, H., Yu, J., Mei, Y., Liu, X., Wang, Y., Zhang, Z., & Bao, F. (2026). TN-AutoRCA: Benchmark construction and agentic framework for self-improving alarm-based root cause analysis in telecommunication networks. In International Conference on Learning Representations (ICLR 2026). https://openreview.net/forum?id=s5mwg63B02

Appendix A – Project Implementation Timeline

Table 11 documents the planned milestone schedule across the 12-week project lifecycle.

Table 11: 12-Week Gantt-Style Project Milestone Schedule

Phase / Task	Planned Start	Planned End	Owner(s)	Milestone Deliverable
Requirements & Literature Survey	Week 1	Week 2	Mohak, Sandarbh, Aarnav	15-paper review & PRD finalized
RCAEval Dataset Ingestion	Week 2	Week 3	Mohak & Sandarbh	RCAEval indexed & partitioned in S3
AWS Free Tier Infrastructure	Week 3	Week 4	Mohak Harsh	Terraform templates for S3/Lambda/DynamoDB
Telemetry Hygiene & SOMP Engine	Week 4	Week 5	Mohak Harsh	Lambda 3-layer filter & SOMP delta
MNSUAD Anomaly Detection Model	Week 5	Week 7	Sandarbh Gupta	PyTorch training scripts & test harness
Service Dependency Graph Model	Week 6	Week 8	Sandarbh Gupta	Trace-derived GNN localization script
Golden Flow & PCAP Verifier	Week 7	Week 8	Sandarbh Gupta	SBI PCAP verifier test scripts
FastAPI Model Context Protocol Server	Week 8	Week 9	Mohak Harsh	MCP tool definitions verified on EC2
React 18 Operator NOC Portal	Week 9	Week 10	Aarnav Mishra	Vite/Tailwind UI & D3.js topology component
AWS Amplify Staging & HITL Modals	Week 10	Week 11	Aarnav Mishra	Amplify staging portal with HITL approval
Automated Testing & Benchmarking	Week 11	Week 12	Mohak, Sandarbh, Aarnav	Test cases T01–T06 evaluated on RCAEval
Final Report & Code Handover	Week 12	Week 12	Mohak, Sandarbh, Aarnav	Final report & review submission